

Infra

Morning Flower and Evening Oath

比赛相关、工具软件、语言基础

目录

第一章 比赛相关	1
1.1 提交前的接口核对	1
1.2 对拍与性能检查	1
第二章 工具软件	2
2.1 编译与运行	2
2.2 Bash 文件与退出状态	2
2.3 Python 参考程序	3
第三章 语言基础	4
3.1 值、引用与调用边界	4
3.2 std::move: 允许移动, 不是立即移动	4
3.3 vector、array、deque 与字符串	4
3.4 set、map 与哈希表	4
3.5 queue、stack 与优先队列	5
3.6 排序、查找与区间算法	5
3.7 pair、tuple、optional 与回调	5
3.8 数值累加与前缀和	6
3.9 complex 与 FFT	6
3.10 C++20 位操作与零边界	6
3.11 随机数与 shuffle 的复现范围	7
3.12 GNU 扩展与查询范围	7
第四章 分册跳转索引	8

第一章 比赛相关

本册是当前模板所用标准库与环境内容的第一版整理，不宣称覆盖整个 C++ 标准库。尚需按实际使用记录补齐未登记的标准库细项、环境版本差异与更多经过执行的示例。

1.1 提交前的接口核对

先对照题面确认下标、闭区间或半开区间、空输入、重边、自环和数值范围。返回值为 0 可能是合法答案，不能一律当作失败。算法模板的数学前提与输出方案约定优先于类型名的直觉。

本库 C++20 模板通常需要 `bits/stdc++.h`、`cassert` 和 `std` 命名空间。GNU 扩展不是标准 C++；若评测环境没有 `libstdc++`，不要假定 `PBDS`、`rope`、`bits/stdc++.h` 可用。整数乘积先转宽类型再乘，不能乘完才强转。断言用于检查调用前提，不能把断言本身当作正确性证明。

1.2 对拍与性能检查

小数据参考应尽量用枚举、直接模拟或不同数学公式。固定随机种子并保存失败输入；先检查样例、空边界、极值、重复值，再测试最大规模。消毒器检查越界与未定义行为，优化版单独测时间。相同机器上的本地耗时也不能直接换算为 OJ 排名。

递归 DFS 保持递归。本机默认线程栈较小时调整本机测试环境，不因此改成显式模拟 DFS。算法的栈深度与题目内存约束仍需匹配。

第二章 工具软件

2.1 编译与运行

以下在 Bash 中运行；源文件和输出文件变量加引号。macOS 的 g++ 可能实际调用 Clang，先用 `-version` 核对。本项目优先采用显式设置的 CXX；未设置时，运行时检查 `g++-16` 是否存在，否则使用 `g++`，不假定机器装有某个版本。CXX 填可执行文件路径或名称，不把编译参数塞进变量。调试与提交使用相同语言标准，GNU 扩展用 GNU 工具链实测。

```
1 if [[ -z "${CXX:-}" ]]; then
2     if command -v g++-16 >/dev/null 2>&1; then
3         CXX=g++-16
4     else
5         CXX=g++
6     fi
7 fi
8 "$CXX" --version
9 src="main.cpp"
10 out="main"
11 "$CXX" -std=c++20 -O2 -Wall -Wextra "$src" -o "$out"
12 ./main < input.txt > actual.txt
13 diff -u expected.txt actual.txt
```

调试构建：

```
1 "$CXX" -std=c++20 -O1 -g -fsanitize=address,undefined \
2     -fno-omit-frame-pointer main.cpp -o debug
3 ./debug < input.txt
```

Linux 先用 `ulimit -Ss` 和 `ulimit -Hs` 查看继承的栈软、硬上限。本库测试只修改测试进程的软上限，目标为请求值与有限硬上限的较小者，绝不修改硬上限；默认请求 524288 KiB，可由 `CPC_TEST_STACK_KIB` 指定正整数。不能无条件设为 `unlimited`，有限硬上限可能不允许。`tools/test_baseline.py` 的收据同时记录计划值与实际值；若硬上限仍不足，须调整执行环境或缩小本地压力输入，不能把失败算通过。macOS 使用 GNU C++ 时本库深递归测试采用链接选项 `-Wl,-stack_size,0x20000000`。这些是本地测试配置，不应盲目加进 OJ 源文件。

2.2 Bash 文件与退出状态

< 重定向输入，> 覆盖输出文件，>> 追加。不要把输入文件名同时作为输出文件名。脚本中可用 `set -euo pipefail` 尽早发现命令失败、未定义变量和管道失败。`diff` 返回 1 通常表示内容不同，不是程序崩溃。

含空格的路径使用双引号；想保留美元符号、反引号等字面内容时，用单引号或带引号的 here-document 结束标记。命令替换不是字符串转义工具，不拼接不可信文本形成命令。

2.3 Python 参考程序

Python 的 `int` 为任意精度整数；`//` 是向下取整，负数时与 C++ 整数除法不同。`pow(a,e,m)` 做模幂；Python 3.8 起，`pow(a,-1,m)` 在可逆时求逆，不可逆会抛异常。`math.isqrt` 返回整数平方根，避免大整数浮点舍入。

```
1 import math
2 import random
3 rng = random.Random(20260913)
4 a = rng.randrange(1, 100)
5 assert (-3) // 2 == -2
6 assert pow(3, -1, 7) == 5
7 assert math.isqrt(10**30) == 10**15
```

大量输入可用 `sys.stdin.buffer`，但一次 `read().split()` 会同时保存全部 token；大样例应估计 Python 对象开销。列表的 `b=a` 是同一个对象，`a[:]` 只复制一层；二维数组用列表推导构造，避免多行共享同一列表。

第三章 语言基础

3.1 值、引用与调用边界

参数 `T` 按值, `const T&` 只读借用, `T&` 允许修改原对象。返回引用必须保证被引用对象仍存活, 不能返回局部变量的引用。结构化绑定 `auto [x,y]` 默认复制, `auto& [x,y]` 引用原元素; 只读遍历常用 `const auto&`。

3.2 `std::move`: 允许移动, 不是立即移动

`std::move(x)` 本身是转换表达式, 使后续重载选择可以使用移动构造或移动赋值; 它自己不搬运数据。对 `const` 对象使用 `move` 不会去掉 `const`, 常见容器因此仍走复制。除有更强保证的特例外, 标准库对象移动后有效但状态未指定: 可销毁、重新赋值, 或调用满足其前提的操作, 不假定内容保持原样或必定为空。

```
1 vector<int> a{1, 2, 3};
2 vector<int> b = std::move(a);
3 a.clear();
4 a.push_back(7);
5 const vector<int> c{4, 5};
6 vector<int> d = std::move(c);
```

上例 `b` 得到原序列, `d` 从 `const c` 复制。调用接受值参数的卷积函数且之后不再需要输入时, 传 `std::move(a)` 可以避免额外副本; 函数参数为 `const` 引用时不会因此取得所有权。返回本地变量通常直接 `return a;`; 多加 `move` 可能失去具名返回值优化机会。依据: [标准草案 move 定义与移动后状态](#)。

3.3 `vector`、`array`、`deque` 与字符串

`vector` 连续存储, 随机访问常数时间, 尾部追加摊还常数时间。`reserve` 只改容量, 不增加元素; `resize` 才改变 `size`。重分配使原指针、引用和迭代器失效; `erase` 使被删位置及之后失效。不要保存 `vector` 节点引用后再 `push` 新节点; 需要跨扩容引用时保存下标并重新访问。[失效规则来源](#)。

`array` 的长度为编译期常量; 局部基本类型元素需用 `{}` 初始化。`deque` 支持两端操作且可下标访问, 但整体不连续; 修改后不要盲目保留迭代器。`vector<bool>` 的元素可能是代理对象, 不等同于普通 `bool` 引用。

`string` 适合字符序列; `substr(pos,len)` 第二个参数是长度。`find` 未找到返回 `npos`, 不能直接转 `int` 再判断。`getline` 读整行, 与 `cin>>` 混用时处理遗留换行。字符串反复头部删除可能线性搬移; 需要复杂序列操作见分册 `rope` 与序列树。

3.4 `set`、`map` 与哈希表

`set/map` 去重, `multiset/multimap` 允许等价键。比较器必须是严格弱序; 不能用 `less_equal` 假装多重集合, 也不直接用不传递的 `epsilon` 比较做排序或树比较。`map` 的下标访问会插入默认值, 只查询用 `find`。

浮点键若含 `NaN`, 普通小于比较也不再满足严格弱序: `NaN` 与不同有限数都可能互不小于, 但这些有限数之间仍有大小关系。几何排序前应遵守有限输入约定, 不能把 `NaN` 或 `epsilon` 当成普通相等键。[严格弱序与等价关系要求](#)。

`multiset` 的 `erase(key)` 会删掉全部等价元素; 只删一个时先 `find`, 再 `erase` 该迭代器。树容器的 `lower_bound` 用成员函数, 通用算法在非随机访问迭代器上可能线性走步。关联容器的键不可原地修改为破坏排序的值。[关联容器要求](#)。

`unordered_map` 的普通查改平均常数时间，最坏可线性；`rehash` 会使迭代器失效。需要容量控制时看其 `reserve/max_load_factor` 接口，不把这些名字照搬到 PBDS。GNU 的 `gp/cc` 哈希表、选型和范围见本册末的分册索引。

3.5 queue、stack 与优先队列

`queue` 先入先出，`stack` 后入先出；取 `front/top` 和 `pop` 前检查非空。标准 `priority_queue` 默认大根堆，`pop` 不返回值，先复制 `top` 再 `pop`。它没有按句柄修改和合并接口。

```
1 priority_queue<int, vector<int>, greater<int>> q;
2 q.push(7);
3 q.push(2);
4 int x = q.top();
5 q.pop();
```

需要修改已入堆的键、删除指定元素或合并时，使用数据结构册的 PBDS 配对堆；不要混用两种 `priority_queue` 的接口或句柄保证。

3.6 排序、查找与区间算法

算法区间通常为半开区间。`sort` 要求严格弱序，`stable_sort` 保留等价元素的原顺序；`nth_element` 只保证第 `k` 位及两侧分区，平均线性，不等于完整排序。`lower/upper_bound` 要求对应有序或分区前提，并使用一致比较规则。[算法要求](#)。

```
1 sort(a.begin(), a.end());
2 a.erase(unique(a.begin(), a.end()), a.end());
3 auto it = lower_bound(a.begin(), a.end(), 7);
4 if (it != a.end()) cout << *it << '\n';
```

`unique` 只合并相邻等价值，返回逻辑终点；它不改容器 `size`。`remove/remove_if` 同理，配合 `erase`；C++20 可用适用容器的 `erase/erase_if`。`reverse`、`rotate`、`merge`、`copy` 的目标区间必须有效，不能把未扩容的 `vector.begin()` 当作足够大的输出区间。

3.7 pair、tuple、optional 与回调

`pair/tuple` 默认按字典序比较，适合把值与唯一编号组合。`optional` 区分无解与合法零值，先判断再解引用。`std::function` 可保存不同可调用对象，但有类型擦除开销；泛型 `lambda` 或模板回调更容易内联。递归 `lambda` 捕获的引用不能活得比被引用对象更久。

`tie(a,b)` 创建引用元组，保存后仍引用原变量，不是值快照。次小生成树用它比较权值与编号；模整数扩展欧几里得用右侧的值 `pair` 先保存旧值，再赋给左侧引用。不能把 `tie(a,b)=tie(b,a)` 当作可靠的交换：右侧也在引用会被改写的变量。

```
1 long long a = 17, b = 5;
2 long long q = a / b;
3 tie(a, b) = pair{b, a - q * b};
4 assert(a == 5 && b == 2);
```

普通数值变量要保存快照可用 `pair{a,b}` 或 `make_tuple(a,b)`，交换则直接 `swap`。依据：[tuple 创建与tuple 赋值](#)。

3.8 数值累加与前缀和

`accumulate` 的初值决定累加类型，64 位求和用 `0LL`；把返回结果装进 `long long` 不会改变中间累加类型。乘积也须先提升，例如 `1LL*a*b`，但仍应保证宽类型可表示结果。[累加定义与算术类型转换](#)。

`partial_sum` 的累加器类型取自输入迭代器的值类型，不由输出容器决定。`SAM` 的长度桶前缀计数用 `int`，前提是总状态数可表示；更大的前缀和应先将输入转为 `long long`，或用显式的宽类型循环。只把输出改为 `vector<long long>` 不能避免 `int` 累加溢出。允许输出起点等于输入起点：

```
1 vector<int> a{1000000000, 1000000000, 1000000000};
2 vector<long long> prefix(a.begin(), a.end());
3 partial_sum(prefix.begin(), prefix.end(), prefix.begin());
4 assert(prefix.back() == 3000000000LL);
```

空区间不写输出；非空时输出区间必须足够大。[partial_sum 定义](#)。`iota` 适合连续编号。`gcd/lcm` 仍有可表示范围前提，最小负数的绝对值应先转宽或转无符号幅值。浮点整数精确范围和 `long double` 有效位数用 `numeric_limits` 查询；几何数值条件另看几何册。

3.9 complex 与 FFT

FFT 使用 `complex<long double>`；`C(re,im)` 分别设置实部、虚部，`real()/imag()` 读取分量，`conj(z)` 取共轭。`abs(z)` 是模长，`norm(z)` 是模长的平方；尤其不要与本库 `RealPlane::norm` 的长度含义混淆。逆变换使用共轭单位根，并按模板约定除以长度。

```
1 using C = complex<long double>;
2 C z(3, 4);
3 assert(z.real() == 3 && z.imag() == 4);
4 assert(conj(z) == C(3, -4));
5 assert(abs(z) == 5 && norm(z) == 25);
```

复数乘法仍是浮点运算，`norm` 也可能溢出，不能因 API 名称而放宽 FFT 的输入范围与舍入误差界。精确整数卷积是否适用浮点版本，应看数学册说明。[分量接口与模长、平方模与共轭](#)。

3.10 C++20 位操作与零边界

`bit_width`、`popcount`、`countl_zero`、`countr_zero` 接受无符号整数类型。`bit_width(x)` 表示表示 `x` 所需的二进制位数，零返回 0；`popcount` 返回 1 的个数。两个 `count-zero` 函数对零都返回该类型的位宽，不是 0。不要把 `countl_zero` 的结果直接当作最高位下标。

```
1 static_assert(bit_width(0u) == 0);
2 static_assert(bit_width(8u) == 4);
3 static_assert(popcount(10u) == 2);
4 static_assert(countr_zero(0u) == numeric_limits<unsigned>::digits);
```

划分树、WaveletMatrix 用 `bit_width((unsigned)max(1,n)-1)` 这类表达式计算层数；其中 `n` 分别取数组长度与去重后的值域大小。先保护零再减一，避免无符号回绕；各模板是否另加一层按接口说明。GCC 的 `__builtin_ctz(0)` 与 `__builtin_clz(0)` 均不能调用，处理 64 位输入还须选对应 `ll` 版本。

`1ULL<<k` 须满足 `k` 非负且小于 `unsigned long long` 的位宽。WaveletMatrix 按 64 位块存储，用 `k=i%64` 得到 0 到 63；`k=0` 时 `(1ULL<<k)-1` 为零掩码，不能用移位 64 构造整块掩码。`bitset` 的大小固定，批量位运算不是与位数无关的常数操作。依据：[bit_width](#)、[位计数](#)、[移位边界](#)与[GCC 内建函数](#)。

3.11 随机数与 shuffle 的复现范围

mt19937 与 mt19937_64 用固定种子，并保持调用顺序、输入和库实现一致，可复现本地对拍。uniform_int_distribution 的两端都包含；rng()%n 一般有偏差。最小覆盖圆使用 shuffle 打乱 vector，它在原半开区间内重排，保留元素及重复次数，接受满足要求的随机位生成器。

固定引擎种子不保证不同标准库的 shuffle 排列或 distribution 输出逐项相同：标准规定 shuffle 的概率要求，未固定具体洗牌算法，分布生成算法也由实现定义。保存失败时的实际输入、种子与工具链；测试排列性质和同一实现内复现，不硬编码某套库产生的排列。随机算法的期望复杂度仍按对应模板说明执行。shuffle 要求与分布算法的实现差异。

3.12 GNU 扩展与查询范围

PBDS 有序树、gp/cc 哈希表、配对堆以及 __gnu_cxx::rope 在所属知识点详细介绍，本册提供集中页码跳转。rope 不叫 std::rope；复制版本的内存成本和已知失败用法不因它是库容器而消失。

第四章 分册跳转索引

请将本册与其他 PDF 放在同一目录。下表页码是分册印刷页码；链接直接指向条目，部分浏览器内置 PDF 阅读器可能不支持跨文件跳转。

内容	所属分册及页码
PBDS 配对堆：修改、合并与句柄	数据结构，第 11 页
PBDS 有序树：排名与第 k 小	数据结构，第 64 页
GNU rope：序列与版本共享	数据结构，第 62 页
gp/cc 哈希表：整数键映射	数据结构，第 1 页
优先队列的最短路应用	图论，第 44 页
vector 与下标：树状数组	数据结构，第 17 页
扩容与下标：可持久化数组	数据结构，第 97 页
比较次序：后缀数组	字符串，第 8 页
数值精度约定：浮点几何	计算几何，第 1 页
移动大数组：NTT 卷积	数学，第 81 页
complex 的平方模与共轭：FFT	数学，第 73 页
位宽与压位计数：小波矩阵	数据结构，第 57 页
tie 与值比较：次小生成树	图论，第 48 页
PBDS 句柄删除：双端优先队列	数据结构，第 12 页
PBDS modify 降键：最短路与路径	数据结构，第 13 页
move 数组：批量逆元与动态模数	数学，第 7 页

算法索引

accumulate, 6
array, 4

Bash, 2
bit-width, 6
bitset, 6

complex, 6
conj, 6

deque, 4

function, 5

g++, 2

lower-bound, 5

map, 4
multiset, 4

nth-element, 5
numeric, 6

optional, 5

pair, 5
partial-sum, 6
PBDS, 7
popcount, 6
pow, 3
priority-queue, 5
Python, 3

queue, 5

random, 7
reference, 4
rope, 7

Sanitizer, 2
set, 4
shuffle, 7
sort, 5
stack, 5
std::move, 4
string, 4

tuple, 5

unique, 5

unordered-map, 4

vector, 4

对拍, 1

提交检查, 1